Engineering Handoff — Cascade Event Instrumentation (v2)

Engineering Handoff — Cascade Event Instrumentation (v2)

Audience: Senior software engineer or systems AI engineer (Staff+) **Estimated effort:** 2.5 weeks total — week 0 for §7 owner decisions, weeks 1–2 for Phases 1–4 (~8.5–10.5 engineering days) **Owner on handoff:** TBD **Author:** David Ancor **Date:** 2026-05-01 (v2 revision) **Supersedes:** `HANDOFF-cascade-events.md` (v1, 2026-05-01) **Related artifacts:** - Methodology paper (live): `/about/methodology` route in this app - Position paper outline: `../../c1v-project-admin-main/position-paper-outline.md` - Strategic context: `../../c1v-project-admin-main/beyond-the-harness.md` - Field vocabulary: `../../c1v-project-admin-main/harness-engineering.md`

Changelog from v1

v2 closes four convention gaps that would have blocked the engineer in week 1, resolves three design decisions previously hidden in §7, and tightens five smaller documentation items.

Convention gaps closed: - §3, §4.1, §4.3 — **Drizzle workflow corrected.** drizzle-kit is broken in this repo; migrations are written as raw SQL. - §3 — **Module Territory Mapping table added.** Maps `extractedData.*` paths to owning modules, prerequisite for Gap 2 middleware. - §3 — **Wall invariant clarified.** The wall is on *evaluator nodes*, not all upstream-modifying nodes; producer nodes may modify upstream territory iff the cascade is logged. - §8, §10 — **Integration-test fixture location named.** Fixture file path specified; if missing, fixture creation is a prerequisite task.

Design decisions resolved (recommendations; owner confirms before Phase 4 starts): - §4.3, §6 — **Cascade dedup** uses key `(run_id, upstream_node, downstream_node, failure_signal_kind)` plus a `count` integer column. Same kind on same pair → one row, count++. Different kinds → separate rows. Was §7 open decision in v1; resolved. - §4.4 — **Async strategy:** meta-analyzer is fire-and-forget. Inline budget < 50ms (enqueue only); diff computation runs in a background worker. Replaces v1's "synchronous, < 200ms p95" framing which was achievable only on small artifacts. - §4.4, §6 — **Three-detector agreement rule:** all three signals (deterministic check, `check_prd_spec`, `wave-e-evaluator`) are logged per cascade candidate; row fires when ≥2 agree; disagreement is recorded in `failure_signal_detail`. New design surface; not present in v1. - §6 — **`cascade_pair_baseline` rollup table** specified for Phase 5 outcome-delta computation. Gives Phase 5 a baseline to compare against; populated as a side-effect of Phase 4 inserts. - §6 — **Postgres-as-storage vs TypeScript-as-API rule** made explicit. Postgres rows are flat; the `CascadeEvent` TypeScript type in `lib/types/cascade-event.ts` is the API surface; one serializer in `lib/db/queries/cascade-events.ts` translates. Eliminates the schema-drift risk between paper figures, dashboard rows, and DB shape.

Documentation tightening: - §5 — Estimates expressed in days (8.5–10.5) and a week 0 added for §7 decisions; total realistic bracket is 2.5 weeks. - §10 — Stub-env test recipe lifted verbatim from `CLAUDE.md` Dev Quirks. Saves the engineer 30 min on day 1. - §1 — Reworded the "does not record" list so the fourth item (consumer absence) is grammatically a reader-not-a-record. - §4.3 — Inline comments added to columns nullable in Phase 4 / populated in Phase 5+ so future archeology has context. - §6 — `vertical` column note added clarifying values for corpus studies (`salesforce_architecture`, `code_review`, etc.) vs default `'system_design'` for c1v's customer projects.

Open decisions remaining in §7: - Diff library (microdiff vs jsondiffpatch) — kept open. - Privacy / data governance for `failure_signal_detail` — kept open. - Dashboard surface (log-only vs UI panel) — kept open. - Historical-runs backfill — kept open; current recommendation: leave NULL. - Postgres column shape (flat vs JSONB) — newly added; current recommendation: flat per the v2 DDL.

---

1. Context (read this first — 3 minutes)

c1v is a multi-agent system that produces complete systems-engineering artifact sets (actors, FFBD, N^2, QFD, FMEA, decision networks, architecture recommendation) from a one-line product brief. It is built on LangGraph with a 19-node DAG (`lib/langchain/graphs/intake-graph.ts`).

The thesis driving this work: beyond model weights (RLHF) and prompts (DSPy), the orchestration topology itself can be optimized. *Cascade events* — moments where a downstream stage forces an upstream stage to redo work — are dense, cheap, interpretable labels for which hand-offs in the graph need correction. We want to instrument them automatically so we can build a dataset, then a meta-agent that proposes topology edits.

The current pipeline already records most of what we need. It tracks per-module revision counters, an append-only revision log with `changed_fields` and `reason`, content-addressed `inputs_hash` on every artifact, and a tamper-detection chain on decision audits. It does **not** record three things, and has no consumer that reads them:

1. **Causal back-link** — which downstream node forced an upstream revision
2. **Structured content diff** — only field names, not values, are captured
3. **Append-only `cascade_events` table** — nothing aggregates the above into the position-paper schema
4. **(no consumer)** — nothing reads the log to surface patterns or propose edits

This handoff is the work to close gaps 1–3 and stand up gap 4 as a reporting-only node (no auto-edits in scope).

---

2. The Goal (definition of done)

After this work ships, every production run of the c1v pipeline will:

1. Emit one `cascade_events` row per detected upstream-redo (deduped per §6), conforming to the schema in §6
2. Populate the causal back-link (`triggered_by_node`) so cascade chains are reconstructible
3. Compute a structural content diff between the prior and new artifact revisions
4. Record three-detector agreement state per cascade (deterministic check / validation / sensor)
5. Update `cascade_pair_baseline` rollups so Phase 5 has baseline metrics to compare against
6. Surface cascade patterns in a basic dashboard (top cascade-prone hand-offs across last N runs)

We are **not** in scope for: meta-agent that proposes topology edits, automatic graph mutation, A/B testing infrastructure for topology experiments. Those are Phase 5+ and require this work to land first.

---

3. Current State (file map)

3.1 Code surface

Concern	File	Lines	Notes
LangGraph DAG definition	`lib/langchain/graphs/intake-graph.ts`	38–76, 452–551	19 nodes, edges, conditional routing
Validation gate (only evaluator)	`lib/langchain/graphs/nodes/check-prd-spec.ts`	120–177, 268–288	**Wall comment at lines 161–163: "validates and reports — does NOT advance"** — preserve this discipline (see §3.3 invariant)
Revision counters per module	`lib/db/schema/project-run-state.ts`	`module_revisions` JSONB field	`{M1: 3, M2: 1, ...}`
Revision log (the seed of the cascade log)	`lib/db/schema/project-run-state.ts`	`revision_log` JSONB field, `RevisionDelta` type	Already captures `module, from_revision, to_revision, changed_fields, changed_by, reason, timestamp`
Content-addressed artifact storage	`lib/db/schema/project-artifacts.ts`	Whole file	Append-only, multiple rows per (project_id, kind), each with `inputs_hash`
Per-decision audit chain	`lib/db/schema/decision-audit.ts` + `lib/langchain/engines/audit-writer.ts`	Whole files	Hash-chain with `hash_chain_prev` for tamper detection
Synthesis observability	`lib/observability/synthesis-metrics.ts`	1–250	Per-node latency, tokens, cost — useful for `outcome_after_edit` later
Wave-E evaluator (sensor-class taxonomy)	`lib/langchain/engines/wave-e-evaluator.ts`	1–168	Tri-state `ready / needs_user_input / failed` — maps directly to `failure_signal.kind`

3.2 Module Territory Mapping (prerequisite for Gap 2)

The middleware in §4.2 must know which `extractedData.*` paths belong to which module so it can attribute *cross-module writes* (the cascade signal). This mapping is the source of truth:

Module	extractedData path	Owning nodes (canonical writers)
M0 (Intake)	`extractedData.intake.*`, `extractedData.projectVision`	`intake-extractor`, `discriminator-intake-agent`
M1 (Scope)	`extractedData.scope.*`, `extractedData.actors`, `extractedData.useCases`	`actors-agent`, `use-case-agent`
M2 (Requirements)	`extractedData.requirements.*`, `extractedData.constants`, `extractedData.openQuestions.requirements`	`requirements-agent`, `module-2/phase-N` agents
M3 (FFBD)	`extractedData.ffbd.*`	`form-function-agent`
M4 (Decision Matrix)	`extractedData.decisionMatrix.*`	`decision-net-agent`
M5 (QFD / HoQ)	`extractedData.qfd.*`, `extractedData.openQuestions.qfdResolved`	`hoq-agent`
M6 (Interfaces)	`extractedData.interfaces.*`	`interface-specs-agent`, `api-spec-agent`
M7 (FMEA early)	`extractedData.fmeaEarly.*`	`fmea-agent` (early pass)
M8 (Risk / FMEA residual)	`extractedData.fmeaResidual.*`, `extractedData.openQuestions.riskResolved`	`fmea-residual-agent`

Rule: if a node not listed as a canonical writer for module M modifies a path under module M's territory, that's a cross-module write → emit a `RevisionDelta` with `triggered_by_node` set to the cross-writing node.

If this mapping is incomplete or wrong, fix it here and update §4.2 in lockstep — the middleware is only as accurate as this table.

3.3 The wall invariant (preserve this)

The wall is on **evaluator nodes**, not on all nodes that may touch upstream territory. Specifically:

- **Evaluator nodes** (`check_prd_spec`, `wave-e-evaluator`, the new `meta-analyzer` from §4.4) — observation-only. Must not modify `extractedData`, `module_revisions`, `revision_log`, or any upstream artifact. Their outputs are reports, not state changes.
- **Producer nodes** — may modify upstream territory if and only if the modification is recorded as a `RevisionDelta` with `triggered_by_node` and `triggered_by_event` populated. This is *the cascade signal*, by design.

If the meta-analyzer ever needs to write to `cascade_events`, that's its OWN territory (downstream-only); it's not breaching the wall.

---

4. The Four Gaps (and what closes each)

Gap 1 — Causal back-link missing on `RevisionDelta`

Today: `RevisionDelta` records `changed_by` (agent name or user) but not which downstream node *forced* the upstream module to revise.

Closing patch: add two optional fields to `RevisionDelta` in `lib/db/schema/project-run-state.ts`:

```typescript
export type RevisionDelta = {
  module: string;
  from_revision: number;
  to_revision: number;
  changed_fields: string[];
  changed_by: string;
  reason: string;
  timestamp: string;
  // NEW
  triggered_by_node?: string;
  triggered_by_event?: 'validation_failure' | 'schema_violation' | 'llm_judge' | 'human_override' | 'cascade';
};
```

Migration note (Drizzle is broken): drizzle-kit is currently non-functional in this repo. Migrations are written as raw SQL files in `lib/db/migrations/` (see `0011_decision_audit.sql` for the convention — manual SQL with explicit RLS / GRANT / index statements). For this gap, write a new migration file that adds the two fields to the relevant column or amends the JSONB shape. Apply via the team's manual-migration command (see `CLAUDE.md` → Dev Quirks).

Acceptance criteria: - New raw-SQL migration file added under `lib/db/migrations/` and applied - Existing rows with no `triggered_by_node` are treated as unknown (NULL semantics in JSONB) — no backfill required - TypeScript compiles cleanly across the codebase

Gap 2 — Middleware to populate the back-link

Today: Nodes write back to `extractedData` for upstream module territory, but the system does not record which node did the writing.

Closing patch: middleware decorator on every node in `intake-graph.ts`. On node entry, snapshot the current `extractedData`. On exit, diff against the snapshot using the §3.2 module territory map. For any module's territory modified by a node *not* listed as that module's canonical writer, append a `RevisionDelta` to `revision_log` with:

- `module` = the modified module (per §3.2 mapping)
- `triggered_by_node` = current node's name
- `triggered_by_event` = derived from how the modification happened (validation gate fail → `validation_failure`, schema reject → `schema_violation`, etc.)

Implementation note: the cleanest place is a wrapper at the LangGraph node-registration site in `intake-graph.ts:38-76`. Wrap each `addNode(name, fn)` call so `fn` becomes `withCascadeMiddleware(name, fn)`.

Modification semantics: "modified" means deep-equal-false on the territory subtree. Identical-content writes (same JSON after canonicalization) do NOT emit a delta. Use the existing `canonicalStringify` from `lib/langchain/graphs/contracts/inputs-hash.ts` to compare.

Acceptance criteria: - Unit tests assert that when node X modifies module Y's territory (per §3.2), a `RevisionDelta` with `triggered_by_node = X` appears in `revision_log` - Identical-content writes produce no `RevisionDelta` (negative test) - Integration test using the §8 cascade fixture: assert at least one `RevisionDelta` per known cascade is recorded with the correct `triggered_by_node` - No regressions in existing pipeline tests

Gap 3 — `cascade_events` table

Today: Aggregating revision data into the position-paper schema requires client-side joins.

Closing patch: new append-only Postgres table (raw-SQL migration; drizzle-kit broken):

```sql
CREATE TABLE cascade_events (
  event_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  run_id            UUID NOT NULL REFERENCES projects(id),
  timestamp         TIMESTAMPTZ NOT NULL DEFAULT now(),
  upstream_node     TEXT NOT NULL,
  upstream_artifact_kind TEXT NOT NULL,
  upstream_state_hash    TEXT NOT NULL,    -- sha256, content-addressed
  downstream_node   TEXT NOT NULL,
  downstream_state_hash  TEXT NOT NULL,
  failure_signal_kind    TEXT NOT NULL CHECK (failure_signal_kind IN
    ('schema_violation','readiness_gate','llm_judge','human_override','exception','validation_failure','cascade')),
  failure_signal_detail  TEXT,

  -- Three-detector agreement state (§4.4). Each is one of: 'fired','clear','not_applicable'.
  detector_deterministic TEXT NOT NULL DEFAULT 'not_applicable',
  detector_validation    TEXT NOT NULL DEFAULT 'not_applicable',
  detector_sensor        TEXT NOT NULL DEFAULT 'not_applicable',

  -- Dedup count: same (run_id, upstream_node, downstream_node, failure_signal_kind) on
  -- repeated firings increments this rather than inserting a duplicate row.
  count             INTEGER NOT NULL DEFAULT 1,

  -- Phase 5+ fields (NULL in Phase 4; populated by meta-agent in later phases).
  corrective_edit_op     TEXT,             -- NULL in Phase 4
  corrective_edit_target TEXT,             -- NULL in Phase 4
  corrective_edit_spec   JSONB,            -- NULL in Phase 4

  outcome_cascade_rate_delta        DOUBLE PRECISION,  -- NULL in Phase 4
  outcome_wallclock_delta_seconds   DOUBLE PRECISION,  -- NULL in Phase 4
  outcome_downstream_reruns_avoided INTEGER,           -- NULL in Phase 4

  labeler           TEXT NOT NULL CHECK (labeler IN ('human','meta_agent','deterministic_check')),
  vertical          TEXT NOT NULL DEFAULT 'system_design',
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Dedup uniqueness constraint matches the §4.4 dedup key.
CREATE UNIQUE INDEX uq_cascade_events_dedup
  ON cascade_events(run_id, upstream_node, downstream_node, failure_signal_kind);

CREATE INDEX idx_cascade_events_run_id ON cascade_events(run_id);
CREATE INDEX idx_cascade_events_upstream_downstream ON cascade_events(upstream_node, downstream_node);
CREATE INDEX idx_cascade_events_timestamp ON cascade_events(timestamp DESC);
CREATE INDEX idx_cascade_events_vertical ON cascade_events(vertical);

-- Append-only enforced via GRANT (RLS policies for INSERT/SELECT only; no UPDATE/DELETE
-- policies). Mirror the pattern from migration 0011_decision_audit.sql.
```

Append-only enforcement: mirror `decision_audit`'s pattern. RLS grants INSERT and SELECT only; the GRANT set excludes UPDATE and DELETE from the app role. The `count++` on dedup is implemented as `INSERT ... ON CONFLICT DO UPDATE SET count = count + 1` — this is allowed because the engineer writes the upsert via the service role, which has UPDATE rights. App-role queries cannot mutate.

Companion rollup table for Phase 5 baselines:

```sql
CREATE TABLE cascade_pair_baseline (
  vertical          TEXT NOT NULL,
  upstream_node     TEXT NOT NULL,
  downstream_node   TEXT NOT NULL,
  baseline_cascade_rate          DOUBLE PRECISION NOT NULL,  -- cascades per run
  baseline_p95_wallclock_ms      DOUBLE PRECISION NOT NULL,  -- p95 wallclock per cascade
  observation_count              INTEGER NOT NULL,           -- runs contributing to baseline
  last_updated      TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (vertical, upstream_node, downstream_node)
);
```

The rollup is updated as a side-effect of `meta-analyzer` inserts (§4.4). Phase 5 reads from this table to compute `outcome_cascade_rate_delta` after a topology edit.

Acceptance criteria: - Drizzle TypeScript schema in `lib/db/schema/cascade-events.ts` matches the SQL above (drizzle is used for *reading* even when migrations are manual) - Rollup schema in `lib/db/schema/cascade-pair-baseline.ts` - Append-only enforced at query layer (no UPDATE / DELETE helpers in `lib/db/queries/cascade-events.ts` ; only `upsert` with `ON CONFLICT count++`) - RLS policies match the pattern used by `project_artifacts` and `decision_audit` (project-scoped read; service-role write) - Single serializer in `lib/db/queries/cascade-events.ts` translates between flat Postgres rows and the `CascadeEvent` TypeScript type from `lib/types/cascade-event.ts` (see §6)

Gap 4 — `meta-analyzer` node (read-only Phase 4)

Today: Nothing consumes `revision_log` or `cascade_events` .

Closing patch: new node in the graph, downstream of `check_prd_spec` and outside any conditional path that affects user-facing output. The node is **fire-and-forget**: it enqueues work to a background job (`lib/jobs/cascade-analyzer-worker.ts`) and returns immediately. The inline budget is **< 50ms** (enqueue only).

Inline (synchronous) work — < 50ms p95: 1. Read all `RevisionDelta` entries from this run's `revision_log` that have `triggered_by_node` populated 2. Build a list of cascade-candidate tuples: `(upstream_module, downstream_node, triggered_by_event)` per delta 3. Enqueue one job per tuple onto the worker queue (Postgres LISTEN/NOTIFY or whatever the team uses) 4. Log: `"Detected N cascade candidates this run; enqueued for analysis"`

Background (asynchronous) work — no latency budget: 1. For each candidate, fetch `(artifact_v_n, artifact_v_n+1)` from `project_artifacts` and compute a structural diff (use `microdiff` per §7 decision; document the choice if changed) 2. **Three-detector agreement check.** For each candidate, evaluate three signals: - **Deterministic check** — was there a real state-hash change between v_n and v_n+1? (computed from the diff) - **Validation check** — did `check_prd_spec` flag this hand-off in this run? (read from observability log) - **Sensor check** — did `wave-e-evaluator` return `failed` or `needs_user_input` on the upstream artifact? (read from observability log)

Each detector's outcome is one of `'fired' | 'clear' | 'not_applicable'` . The cascade is logged as a real cascade if **≥2 detectors fired**. Record all three signals on the row regardless (even when only 1 fires, for later analysis of detector disagreement). 3. Insert / upsert one `cascade_events` row per cascade, with `labeler = 'deterministic_check'` and Phase 5+ fields NULL. Use `ON CONFLICT (run_id, upstream_node, downstream_node, failure_signal_kind) DO UPDATE SET count = count + 1` for dedup. 4. Update `cascade_pair_baseline` for the corresponding `(vertical, upstream_node, downstream_node)` triple — increment `observation_count` , recompute `baseline_cascade_rate` as a moving average. 5. Surface a summary in the synthesis observability log: `"Run X: detected N cascades; 2-of-3 agreement on K of them; top hand-off: M5→M3"`

Acceptance criteria: - Meta-analyzer node added to `intake-graph.ts` after `check_prd_spec` , before terminal nodes - Node returns within 50ms p95 inline (enqueues only) - Worker processes the queue and writes to `cascade_events` + `cascade_pair_baseline` - Node never modifies `extractedData` or any upstream state (preserves the §3.3 wall) - Idempotent on re-runs (the dedup unique index on `(run_id, upstream_node, downstream_node, failure_signal_kind)` enforces this) - Failure of meta-analyzer (or its worker) never aborts the run (try/catch + Sentry log)

---

5. Phasing & Estimates

Phase	Gap	Estimate (senior eng)	Blocking?
Week 0	Owner decisions in §7	2–3 days (collaborative)	Blocks 4
1	Schema patch + manual SQL migration (Gap 1)	0.5 day	Blocks 2
2	Middleware + module-territory map alignment (Gap 2)	2 days	Blocks 4
3	`cascade_events` + `cascade_pair_baseline` tables (Gap 3)	1 day	Blocks 4
4	`meta-analyzer` node + background worker (Gap 4)	5–7 days	—
Total	**8.5–10.5 engineering-days + week 0 = ~2.5 weeks calendar**		
5 (out of scope here)	Topology-edit proposer	4–6 weeks	Requires n=10+ dataset from Phase 4

Phases 1 and 3 can be done in parallel during week 1. Phase 2 must follow Phase 1. Phase 4 must follow Phases 2 and 3 and the week-0 decisions in §7.

---

6. Reference: Cascade Event Schema (canonical)

This is the canonical TypeScript shape (the API surface). Postgres storage is flat (see §4.3 DDL); a single serializer in `lib/db/queries/cascade-events.ts` translates between the two. Mirrors §4 of the position-paper outline; keep the three surfaces (paper / TS type / Postgres DDL) in sync if you modify any.

```typescript
// lib/types/cascade-event.ts — the API surface
type CascadeEvent = {
  event_id: string;                       // UUID
  run_id: string;                         // foreign key to projects.id
  timestamp: string;                      // ISO-8601
  upstream_node: string;                  // e.g. "M3"
  upstream_artifact_kind: string;         // e.g. "fmea_early_v1"
  upstream_state_hash: string;            // sha256 from project_artifacts.inputs_hash
  downstream_node: string;
  downstream_state_hash: string;
  failure_signal: {
    kind: 'schema_violation' | 'readiness_gate' | 'llm_judge'
        | 'human_override' | 'exception' | 'validation_failure' | 'cascade';
    detail?: string;
  };
  detector_agreement: {                   // §4.4 three-detector rule
    deterministic: 'fired' | 'clear' | 'not_applicable';
    validation:    'fired' | 'clear' | 'not_applicable';
    sensor:        'fired' | 'clear' | 'not_applicable';
  };
  count: number;                          // dedup count; same key fired N times → count = N
  corrective_edit?: {                     // Phase 4: NULL; Phase 5: populated by meta-agent
    operation: 'add_feedback_edge' | 'insert_gate' | 'reorder' | 'split' | 'merge' | 'promote_state';
    target: string;
    spec: string | object;
  };
  outcome_after_edit?: {                  // Phase 5+
    cascade_rate_delta: number;
    wallclock_delta_seconds: number;
    downstream_reruns_avoided: number;
  };
  labeler: 'human' | 'meta_agent' | 'deterministic_check';
  vertical: string;                       // see §6.1 below
};
```

6.1 Dedup key

The unique index in §4.3 — `(run_id, upstream_node, downstream_node, failure_signal_kind)` — is the dedup key. Same kind on the same hand-off in the same run → `count++` on a single row. Different `failure_signal_kind` on the same hand-off → separate rows. This preserves both the count signal (how often did this hand-off cascade?) and the kind diversity (did it cascade for different reasons?).

6.2 `vertical` column values

Value	When to use
`'system_design'`	c1v's customer projects (default)
`'salesforce_architecture'`	Corpus study using Salesforce industry-cloud reference architectures
`'code_review'`	Code-review-vertical experiments per position-paper §6.1
Other	Use a clear, lowercase, snake_case identifier; document in `harness-engineering.md` glossary

The vertical column is the filter that powers cross-vertical transfer claims in the paper. Set it correctly at insert time — backfill is painful.

6.3 Postgres ↔ TypeScript serializer

Postgres stores the row flat (`failure_signal_kind`, `failure_signal_detail` as separate columns). The TypeScript type nests them (`failure_signal: { kind, detail }`). The single serializer at `lib/db/queries/cascade-events.ts` is the authoritative translator. Do not write a second translator anywhere in the codebase. If a third surface needs the data (dashboard, paper-figure exporter), it consumes the serializer's output, not the raw row.

7. Open Decisions to Escalate to Owner (week 0)

These are choices the engineer should *not* make alone. Surface them in week 0, before writing code.

1. **Diff library.** `microdiff` (smaller, simpler) vs `jsondiffpatch` (richer output, larger). Recommend `microdiff` for Phase 4; revisit if Phase 5 needs richer signal.
2. **Privacy / data governance.** `cascade_events` rows contain artifact state hashes and node names. They do **not** contain user PII directly, but `failure_signal_detail` could (free-text from LLM-as-judge). Confirm with owner whether to:
   - a. Restrict `failure_signal_detail` to a controlled vocabulary
   - b. Scrub free-text via LLM call before insert
   - c. Allow free-text but RLS-gate read access more tightly than other tables
3. **Dashboard surface.** §4.4 Phase 4 spec says "surface a summary in observability log." Owner may want a UI panel in the synthesis dashboard. Out of scope here unless explicitly added.
4. **Backfill of historical runs.** Existing `revision_log` data does not have `triggered_by_node`. Decision: leave historical as `NULL` (current plan) or attempt heuristic backfill? Recommend leave; the n=10 dataset starts from Phase 4 ship date.
5. **Postgres column shape — flat vs JSONB.** v2 DDL flattens nested objects (`failure_signal_kind` + `failure_signal_detail` as columns) for query performance and CHECK-constraint clarity. Alternative: store the whole `failure_signal` as JSONB and let the serializer do the work. Recommend flat (current v2 DDL); JSONB is acceptable if owner has a specific reason. Decide before Phase 3.

8. Testing Strategy

Layer	Coverage required
Unit	Middleware diff logic, schema validators, dedup key generation, serializer round-trip
Integration	Run a known cascade-producing intake through the pipeline; assert `cascade_events` table rows match expected count, structure, and detector-agreement state
Regression	Existing pipeline tests must pass unchanged
Performance	Meta-analyzer inline node returns < 50ms p95 (enqueue-only); worker latency tracked but not budgeted
Observability	Sentry / Pino logs include cascade detection counts per run; rollup table updated within 5 minutes p95

8.1 Reproducible cascade fixture

The c1v methodology paper documents three concrete cascade events from a real self-application run (M4 premature scoring, M5 EC underspecification, M7 terminal-FMEA).

Fixture location: - `__tests__/fixtures/cascade-self-application-v1.json` — full intake payload that reproduced the three cascades

If fixture file does not exist (highly likely; check first), creating it is a **prerequisite task** before Gap 2 integration tests: - Owner provides the original intake from the methodology-paper run, OR - Engineer generates a synthetic fixture by replaying the three cascades from the paper's documented inputs/outputs, OR - Mark the integration test as `xfail` for Phase 2 and complete it after fixture exists (track separately)

Expected fixture output: ≥3 `cascade_events` rows with the documented upstream/downstream pairs and `failure_signal_kind` values matching the paper's §5 table:

Cascade	Upstream → Downstream	failure_signal_kind
C1	M4 → M5	`readiness_gate`
C2	M5 → M3	`validation_failure`
C3	M7 → M1	`cascade`

8.2 Stub-env recipe (lifted from CLAUDE.md Dev Quirks)

Tests run from `apps/product-helper/` with format-valid stub env vars. Minimum passing recipe:

```
POSTGRES_URL=stub \
AUTH_SECRET=stubstubstubstubstubstubstubstubstub \
STRIPE_SECRET_KEY=sk_test_stub \
STRIPE_WEBHOOK_SECRET=whsec_stub \
OPENROUTER_API_KEY=sk-or-stub \
BASE_URL=http://localhost:3000 \
npx jest <path>
```

(`ANTHROPIC_API_KEY` is now optional post-OpenRouter migration 2026-04-30.) The old all-`stub` recipe fails at import time because `lib/config/env.ts` enforces prefix and length validation. For scripts that hit the live DB, `set -a; source .env.local; set +a; …` instead.

---

9. Out of Scope (do not gold-plate)

- Meta-agent that *proposes* topology edits (Phase 5)
- Automatic graph mutation or feature-flagged A/B of topologies
- Cross-vertical generalization (this is the system-design vertical only)
- UI changes beyond an optional log line
- LLM-as-judge sensor implementation (separate work; tracked elsewhere)
- Backfill of historical `revision_log` data into `cascade_events`
- Real-time dashboard updates (rollup is eventually consistent within 5 minutes)

If any of these become tempting during implementation, stop and check with the owner.

---

10. First Day Checklist

1. Read this document end to end (~30 min)
2. Read `lib/langchain/graphs/intake-graph.ts` and `lib/langchain/graphs/nodes/check-prd-spec.ts` (~45 min)
3. Read `lib/db/schema/project-run-state.ts`, `lib/db/schema/project-artifacts.ts`, `lib/db/schema/decision-audit.ts` (~30 min)
4. Run the existing test suite using the §8.2 stub-env recipe (lifted from `CLAUDE.md` so you don't have to find it). Confirm green baseline.
5. Run a sample synthesis end-to-end locally; trace one node's invocation through `synthesis-metrics.ts` to understand the observability surface
6. Verify the §3.2 module territory map against the actual code — does `extractedData.qfd.*` actually live where this doc says it does? File any discrepancies as v2.1 patches to this doc.
7. Confirm the §8.1 fixture file exists. If not, raise it as a prerequisite blocker.
8. Open a thread with the owner on the five week-0 decisions in §7 before writing any code.
9. Start Phase 1 (schema patch) — smallest possible change, lowest risk, unblocks Phase 2.

---

11. Why This Matters (one paragraph for context — read on day one)

This work converts an existing audit log into a research-grade dataset that demonstrates a novel claim: that multi-agent orchestration topologies can be optimized using inter-stage cascade events as a gradient signal — without a learned reward model and without retraining the underlying LLM. The c1v product already runs in production. The methodology paper at `/about/methodology` is the n=3 hand-labeled proof of concept. After this work ships, every production run becomes a labeled training example. After 10 runs, we have a dataset. After 100, we have a meta-agent. After 1,000, we have a paper at NeurIPS or ICLR. This handoff is the bridge from product to research without breaking the product.

The technical work is small. The strategic leverage is large. Treat it accordingly.